

AI Agents 生成式人工智能工具整合與工作流程應用

原理、設計模式與警務應用實戰

依據 Prompt Engineering Guide: Agents

工作流 vs 代理

工具、記憶、脈絡與驗證

參考：<https://www.promptingguide.ai/agents>

教學定位與使用方式

- **定位**：把 LLM 從「問答工具」提升到「可規劃、可使用工具、可追蹤狀態的任務系統」。
- **學習節奏**：先用 planning 拆任務並形成 workflow，再看 workflow patterns、tool use 與 function calling，最後用 context 與 memory 控制整體行為。
- **警務使用原則**：agent 可協助檢索、整理、草擬與檢核，但不取代法律判斷、證據審查或主管審批。
- **資料治理**：涉及個資、案情、內部規程時，必須有授權、最小揭露、日誌、審計與人員覆核。

新手入門操作路徑

如果學員沒有 LLM 或 prompt engineering 背景，建議先做 copy-paste walkthrough，再進入完整練習。

使用檔案

- `data/ai-agents/beginner_walkthrough_zh-TW.md`

三個入門練習

Walkthrough	學到什麼
1. Single → Workflow	為什麼要拆步驟、列來源、保留 open questions
2. 普通回答 → Tool Use + Function Call	為什麼天氣、資料庫、schema 需要工具
3. 普通 SOP 問答 → Context + Memory	為什麼要有規則、記憶、error handling

每個 walkthrough 都包含 baseline prompt、常見錯誤、改良 prompt、預期輸出和 `TODO` 修改。

Topic 1 | Introduction to AI Agents

先建立共同語言：什麼是 agent？為什麼需要 agent？

本節重點：agent 不是「比較聰明的聊天機器人」，而是結合規劃、工具、狀態與覆核的任務系統。

什麼是 AI Agent ?

Prompt Engineering Guide 的核心定義

AI agent 是由 LLM 驅動、能採取行動並自主解決複雜任務的系統。

- 不只是生成文字，而是能**規劃下一步**
- 能使用外部工具、API、資料庫或軟件
- 能保存與取回任務過程中的資訊
- 能根據新觀察調整策略

參考：Introduction to AI Agents

關鍵轉變

傳統 LLM	AI Agent
回答一次問題	迭代完成任務
主要依賴模型內部知識	可接外部資料與工具
使用者手動控制流程	系統可自選步驟
難以追蹤長任務	可加入記憶與任務狀態

Standard Prompt 跟 Agent 有什麼差別？

先分清楚：不是所有 LLM 任務都等於 agent

比較面向	Standard Prompt	Agent
任務型態	一次輸入，一次輸出	可多步驟迭代完成任務
流程控制	使用者自己想好步驟	系統可規劃、重排或選擇下一步
資訊來源	主要靠 prompt 內文字與模型知識	可接工具、API、檢索、資料庫
中間狀態	通常不保留	可保存 task state、memory、tool log
錯誤處理	答錯就重問一次	可檢查、重試、升級人工覆核
適合場景	改寫、摘要、翻譯、單次分類	長任務、跨來源查核、多步驟決策支援

Discussion：模型會自己分步，為什麼仍不等於 Workflow？

教學上不要用「模型腦內是否分解任務」來分類

可能發生的事

現代 LLM 即使用一個簡單 prompt，
也可能在內部先做：

- 任務拆解
- 隱性排序
- 自我檢查
- 草擬後再改寫

但這些步驟通常：

- 使用者看不到
- 教師無法驗證
- 每次未必可重現
- 不能直接審計合規性

教學上的判準

如果流程只存在模型腦內，仍可先視為
`simple prompt`。

只有當流程被外顯成：

- 明確步驟
- 可修改順序
- 可檢查中間輸出
- 可配合工具、狀態、日誌

才適合教成 `workflow` 或 `agent`。

區分重點不是「模型有沒有想很多步」，而是「流程是否可見、可控、可重現、可審計」。

教學時怎樣區分 Simple Prompt、Workflow、Agent？

用可觀察控制面來分，不用猜模型內部推理

類型	教學上的判準	學員看得到什麼
Simple Prompt	一次輸入，一次輸出；中間流程不外顯	只有 prompt 與答案
Workflow Prompt	步驟由教師或開發者先寫出來	可看到固定步驟與每步要求
Agent	系統可根據狀態選下一步，並常配合工具與記憶	可看到任務狀態、工具使用、例外處理

即使模型在背後有 internal decomposition，
只要那個流程不能被教、不能被查、不能被穩定重做，課堂上就不應把它算成 workflow。

同一個任務，Prompt 與 Agent 的做法差在哪裡？

差別不只是答案，而是中間有沒有任務處理過程

Standard Prompt

使用者：

「根據以下活動資料，寫一段明天社區防騙宣傳 briefing。」

- 模型直接生成答案
- 好處：快、簡單、容易教
- 風險：可能把舊資料、漏資料、推測語句一起寫進去

Agent

系統先做：

1. 檢查 briefing 需要哪些欄位
2. 如涉及天氣或最新活動安排，先呼叫工具
3. 缺資料時列出 `open_questions`
4. 只用 confirmed facts 生成草稿
5. 輸出前附上來源與待人工確認項目

重點不是 agent 「比較聰明」，而是 agent 會先處理任務，再決定怎樣回答。

警務情境例子：什麼只要 Standard Prompt？什麼才需要 Agent？

先看任務需求，不要先看名稱是不是「AI」

任務	用 Standard Prompt 就夠？	為什麼
把公告改寫成較易明版本	可以	任務短，輸入明確，沒有外部查核要求
將會議紀錄整理成三點摘要	可以	只是重組已提供內容
把投訴文本初步分成 3 類	多數可以	若只是初步分類，可先用固定 prompt
產生活動 briefing 草稿，且要補最新天氣	不夠	需要工具查最新資料
回答 SOP 程序問題，還要指出版本與段落	不夠	需要檢索指定規程，不能靠常識回答
檢查案件摘要是否缺欄位，並要求補件	更適合 agent	需要逐步檢查、追蹤狀態與缺漏

不要為了「高級」而用 Agent

先判斷有沒有必要增加自主性

用 standard prompt 較好

- 已有完整資料，沒有外部查詢需求
- 結果只需一次輸出
- 不需要保留中間狀態
- 不需要工具權限
- 容錯成本低，答錯可直接重問

用 agent 較好

- 任務要分幾步做才穩定
- 中間需要查工具或資料源
- 要記錄來源、狀態與例外
- 需要 validation、retry 或人工覆核
- 任務不是「寫一段字」，而是「完成一個流程」

課堂設計上，應先教學員判斷「需不需要 agent」，再教他們怎樣做 agent。
否則容易把所有問題都錯誤升級成 agent 問題。

一開始怎樣使用 Agent ？

確定需要 agent 之後，才進入設計

1. 定目標

講清楚要交付什麼，不是只說「幫我處理」

2. 定資料

指定可用資料、不可用資料、最新性要求

3. 定工具

哪些情況必須查天氣、查 SOP、查資料庫

4. 定規則

輸出格式、禁止推測、遇到不足時要停下來

5. 定覆核

哪些內容只可做草稿，最後仍需人工批准

不要一開始就把任務寫成「自己想辦法完成」。

先給目標、來源、工具邊界與停止條件，agent 才會變成可控系統，而不是不可預測的黑箱。

Topic 1 深入：Agent 不是單一功能

Agent 更像一個「有目標的任務執行系統」，由模型、工具、狀態、規則與人員覆核共同構成。

判斷一個系統是否像 agent

- 是否能拆解任務，而不只是回答一次？
- 是否能根據中間結果改變下一步？
- 是否能呼叫工具取得新資訊或執行動作？
- 是否有任務狀態、記憶或紀錄？
- 是否有停止、升級或覆核機制？

警務角度

一個「案件摘要工具」未必是 agent。
如果它能讀資料、檢查缺漏、追蹤任務、要求補資料、生成覆核清單，才開始接近 agentic system。

Topic 1 深入：自主程度分級

層級	系統行為	適合任務	風險
L0 單次 LLM	一問一答	改寫、摘要、翻譯	易幻覺
L1 Workflow	固定步驟	穩定行政流程	太僵硬
L2 Tool Assistant	按規則用工具	查 SOP、查公開資料	工具誤用
L3 Agent	自主規劃與用工具	開放研究、跨來源整理	難預測
L4 Deep Agent	長任務、委派、記憶	長期研究、複雜報告	治理成本高

警務教學應先從 L1–L2 開始，確認日誌、權限、驗證與人工覆核成熟後，再討論 L3–L4。

為什麼需要 Agent ？

一般 LLM 擅長翻譯、摘要、改寫、草擬電郵等**單步或窄範圍任務**。
但複雜任務通常需要多步推理、外部資訊、狀態追蹤與重試。

例：做一份治安熱點初步分析，不只是「寫報告」。

它可能需要讀取資料、檢查缺漏、查詢地點、比對時段、生成圖表、列出不確定性，再產生可覆核的摘要。

多步驟

外部資料

工具執行

狀態追蹤

人工覆核

Topic 2 | Prompt Control: Single, Workflow, Dynamic, Hybrid

同一個任務，用四種 prompt 比較控制方式

本節不是先背概念，而是讓學員用同一份資料做四次實驗：

single prompt、workflow prompt、dynamic workflow prompt、hybrid prompt。

四種 Prompt 的差別

Prompt 類型	步驟由誰決定？	遇到資料陷阱時會怎樣？	適合用途
Single prompt	沒有明確步驟	可能直接寫進草稿	快速草稿、低風險摘要
Workflow prompt	使用者 / 開發者固定	按固定檢查表標記問題	可控、可教、可審計任務
Dynamic workflow prompt	模型動態拆解	根據狀態決定下一個檢查動作	探索、資料不完整任務
Hybrid prompt	外層固定，局部交給模型	外層不變，研究步驟可動態追查	高風險但需要彈性的任務

Workflow Prompt 怎樣寫？

Workflow prompt 的核心是：**使用者先把步驟固定，模型只負責按順序執行。**

你是 workflow executor。請嚴格按指定步驟執行，不得跳步。

任務：

{{task}}

固定 workflow：

1. {{step_1}}：輸入是什麼？輸出是什麼？
2. {{step_2}}：輸入是上一步結果，輸出是什麼？
3. {{step_3}}：檢查什麼？失敗時怎樣處理？

規則：

- 不得自行新增或刪除步驟。
- 每一步必須輸出 step result。
- 若資料不足，寫入 open_questions，不得推論。

輸出：

Step Log

Intermediate Results

Final Output

Open Questions

Workflow Prompt 寫作重點

要寫清楚	原因	例子
每一步名稱	方便檢查模型有沒有跳步	Source Extraction
每一步輸入	避免模型使用未授權資料	只使用 event_briefing_data
每一步輸出	讓中間結果可審計	Claim Table
失敗處理	防止模型硬答	資料不足 → open_questions
最終格式	方便比較與評分	Briefing Draft + Human Review

何時用？

當你已經知道正確流程，而且希望模型穩定跟著 SOP 做。

Dynamic Workflow Prompt 怎樣寫？

Dynamic workflow prompt 的核心是：**使用者給任務邊界，模型根據目前狀態決定下一步。**

你是 dynamic workflow executor。

任務：
{{task}}

可用資料：
{{data}}

動態決策規則：

- 每一輪先輸出 current_state。
- 再選擇一個 next_action。
- 每個 action 必須說明 reason。
- 每輪 action 後更新 task_memory。
- 最多執行 {{max_actions}} 個 action。
- 若資料不足，停止並輸出 Need Human Review。

允許的 actions：

- inspect_sources
- check_conflicts
- update_memory
- draft_answer
- request_human_review

Dynamic Workflow 寫作重點

Dynamic prompt 不是叫模型自由發揮，而是給它一個「可選 action 的清單」。

要寫清楚	原因	例子
可用 actions	限制模型可以做什麼	<code>inspect_sources</code> , <code>draft_answer</code>
action limit	防止一直查下去	最多 5 個 action
state format	讓每一步可追蹤	<code>current_state</code> , <code>next_action</code>
memory update	避免忘記限制	更新 <code>open_questions</code>
stop condition	資料不足時停下來	<code>Need Human Review</code>

何時用？

當資料有缺口、來源衝突，或你不知道一開始應該查哪一步。

Hybrid Prompt 怎樣寫？

Hybrid prompt 的核心是：外層 workflow 固定，只有指定步驟允許動態決策。

你是 hybrid workflow executor。

外層 workflow 固定：

1. Scope Check
2. Dynamic Research
3. Memory Update
4. Draft
5. Verification
6. Human Review

規則：

- Step 1, 3, 4, 5, 6 必須按順序執行。
- 只有 Step 2 Dynamic Research 可自行選擇 next_action。
- Dynamic Research 最多 {{max_actions}} 個 action。
- Draft 只可使用 confirmed_facts。
- Verification 不可跳過。

Hybrid Prompt 寫作重點

Hybrid 的難點是寫清楚「哪裡可以動，哪裡不可以動」。

部分	固定還是動態？	寫法
Scope Check	固定	確認任務是否允許、資料是否公開
Dynamic Research	動態	可自行選 action，但有 action limit
Memory Update	固定	必須更新指定 memory schema
Draft	固定	只根據 <code>confirmed_facts</code> 寫
Verification	固定	檢查 unsupported claims / open questions
Human Review	固定	高風險或資料不足時必須標記

何時用？

當任務需要彈性追查，但正式輸出又必須保持可控和可審計。

實驗資料：四種 Prompt 使用同一份輸入

任務

為「海濱文化週末」整理公開資料前 briefing 草稿。

使用資料

- `data/ai-agents/event_briefing_task_zh-TW.md`
- `data/ai-agents/event_briefing_data_zh-TW.md`

比較重點

- 有沒有先拆步驟？
- 有沒有保留來源與 open questions？
- 有沒有避免推測？
- 有沒有人工覆核條件？
- 有沒有把社交媒體、舊版頁面、新聞推測誤寫成官方結論？

資料本身不應暗示流程。四種實驗的差別來自 prompt 如何控制模型行為。

實驗 1：Single Prompt

請根據以下資料，為「海濱文化週末」寫一份簡前 briefing 草稿。

資料：

{{event_briefing_task}}

{{event_briefing_data}}

請包含：

- 活動基本資料
- 交通與天氣注意事項
- 部署建議

觀察重點

- 是否直接寫答案？
- 是否把未確認資料寫成結論？
- 是否漏掉來源、限制或人工覆核？

Single Prompt：預期輸出形態

Single prompt 通常會直接產生一份流暢草稿；問題是它可能把弱來源混入正式語氣。

海濱文化週末將於 2026-06-20 至 2026-06-21 在南灣湖一帶舉行。
活動時間為 15:00-21:00，兩日周邊道路將封路至午夜。
預計活動期間人流會大幅增加，6/21 可能增設入口檢查點。
如兩日均有驟雨，應準備雨天人流分流方案。

這份結果的問題

- 15:00-21:00 來自 2025 舊版頁面，不應覆蓋 2026 官方資料。
- 兩日封路至午夜 來自社交媒體截圖，不是官方公告。
- 6/21 增設入口檢查點 來自志願者群組，不是正式公告。
- 兩日均有驟雨 只來自舊天氣摘要，不是最新預報。
- 草稿沒有列出 open questions 或 human review。

Single Prompt 的常見問題

優點

- 快速
- 容易使用
- 適合低風險草稿

風險

- 不一定先查來源
- 不一定保留 open questions
- 容易把「可能」寫成「已確認」
- 難以審計中間步驟

實驗 2：Workflow Prompt

你是 workflow executor。請嚴格按以下步驟執行，不得跳步。

任務：為「海濱文化週末」整理公開資料勤前 briefing 草稿。

固定 workflow：

1. Scope Check：確認只使用公開資料。
2. Source Extraction：從提供資料抽取日期、地點、交通、天氣、來源。
3. Claim Table：把每個 key claim 對應到 source_id。
4. Draft：只根據已確認 claim 寫草稿。
5. Verification：列出 open_questions、unsupported_claims、human_review_required。

輸出：

Step Log

Claim Table

Briefing Draft

Open Questions

Human Review

Workflow Prompt：預期輸出形態

Workflow prompt 應該先輸出中間表格，再寫草稿；它的結果不一定最完整，但較容易檢查。

```
## Step Log
- Scope Check: public_only confirmed
- Source Extraction: SRC-A, SRC-B, SRC-C, SRC-D reviewed
- Claim Table: completed
- Draft: generated from supported claims only
- Verification: open_questions found

## Claim Table
| claim | source_id | status |
|---|---|---|
| 活動日期為 2026-06-20 至 2026-06-21 | SRC-A | confirmed |
| 6/20 有臨時交通安排 | SRC-B | confirmed |
| 6/21 是否延續交通安排 | none | open_question |
| 兩日均封路至午夜 | SRC-E | unsupported / do_not_conclude |
| 活動時間為 15:00-21:00 | SRC-F | outdated_source |

## Open Questions
- 6/21 交通安排是否延續未能確認。
- 最新天氣預報需接近活動日期再查。
```

Workflow Prompt 的重點

Workflow prompt 是使用者或開發者先定義流程。

步驟固定：

P1 → P2 → P3 → P4 → P5

模型任務：

- 按順序完成
- 不自行改流程
- 每一步輸出可檢查結果
- 失敗時按 fallback 處理

適合

- 教學練習
- SOP 流程
- 高風險輸出前的固定檢查

實驗 3：Dynamic Workflow Prompt

你是 dynamic workflow executor。請根據任務狀態自行決定下一步。

任務：

為「海濱文化週末」整理公開資料勤前 briefing 草稿。

可用資料：

{{event_briefing_task}}

{{event_briefing_data}}

Dynamic workflow rules:

- 每次先輸出 current_state。
- 再決定 next_action。
- 若資料不足，寫入 open_questions，不得補作推測。
- 若來源不足，不得寫入 confirmed_facts。
- 最多執行 5 個 action；完成後輸出 briefing draft。

輸出：

Current State

Next Action

Updated Memory

Final Draft 或 Need Human Review

Dynamic Workflow 的重點

Dynamic workflow 是 runtime 動態決策。

固定 workflow :

P1 → P2 → P3 → P4 → P5

Dynamic workflow :

看完目前狀態後，再決定下一步。

下一步可能是查來源日期、追查官方公告、更新 memory、或停止。

和 Workflow 的關鍵差別

Workflow 是「照指定路線走」；Dynamic workflow 是「按現場狀態選下一步」。

Dynamic Workflow：同一份資料會怎樣行動？

Current State

- SRC-B 只確認 6/20 臨時交通安排。
- SRC-E 說兩日封路至午夜，但沒有官方來源。
- SRC-F 是 2025 舊版活動頁，時間與本次資料不一致。
- WEATHER-SIM-OLD 是 10 日前天氣資料。

Agent 可決定：

1. 把 SRC-F 標記為 `outdated_source`。
2. 把 SRC-E 標記為 `unsupported / do_not_conclude`。
3. 追查官方交通資料是否提到 6/21。
4. 若沒有最新天氣工具，寫入 `open_questions`。
5. 更新 `task_memory` 後才產生草稿。

重點不是「模型比較聰明」，而是 prompt 允許模型在限制內改變下一步。

Dynamic Workflow：預期輸出結果

Dynamic workflow 的結果應該看到「行動路線改變」，而不只是多一張表。

```
## Action Log
Action 1: inspect_source_dates
Finding: SRC-F is a 2025 page. Do not use for 2026 event time.

Action 2: verify_traffic_claim
Finding: official traffic source confirms 6/20 only. 6/21 remains unknown.

Action 3: assess_non_official_sources
Finding: SRC-E and SRC-G are not official. Move to unsupported_claims.

Action 4: check_weather_freshness
Finding: WEATHER-SIM-OLD is too early. Latest weather required.

## Final Decision
Generate draft, but mark Need Human Review = true.
```

和 Workflow 的結果差別

Workflow 是按固定表格檢查；Dynamic workflow 是發現問題後，自己決定下一個檢查動作。

Dynamic Workflow：優勢與代價

面向	Dynamic workflow 的優勢	需要限制的原因
資料不完整	可先找缺口，再決定查什麼	可能查太多、拖太久
來源衝突	可先處理衝突來源，再寫草稿	可能忽略原本交付格式
新發現	可新增 action，例如改查官方公告日期	可能行動不一致
審計	可輸出 action log / memory	必須要求記錄每一步

必要 guardrails

最多 5 個 action
 只可使用指定資料或指定工具
 每次 action 後更新 memory
 資料不足時停止並列 Need Human Review

實驗 4：Hybrid Prompt

你是 hybrid briefing assistant。

外層 workflow 固定：

1. Scope Check
2. Dynamic Research
3. Memory Update
4. Draft
5. Verification
6. Human Review

只有 Step 2 Dynamic Research 可由模型動態決策。
其他步驟必須按固定順序執行。

Dynamic Research 規則：

- 可自行決定先整理官方資料、交通、天氣或新聞背景。
- 每次 research action 後必須更新 task_memory。
- 不得直接輸出 final briefing。

輸出：

```
## Workflow Step Log
## Dynamic Research Actions
## Task Memory
## Briefing Draft
## Verification Report
```

Hybrid Prompt 的重點

Hybrid 是實務常見設計：固定高風險步驟，保留局部彈性。

固定 Workflow	Dynamic Research 動態部分
Scope Check	查哪些公開資料
Memory Update 格式	調整查詢方向
Draft 格式	找出 open questions
Verification	整理初步 findings
Human Review	不可跳過

Hybrid：它和 Dynamic Workflow 不同在哪？

問題	Dynamic Workflow	Hybrid
誰決定整體流程？	模型可動態安排	使用者固定外層 workflow
哪裡可以動態？	整個任務都可能動態	只有指定步驟可動態
能否跳過 Verification？	若 prompt 沒限制，可能會	不可以，固定步驟必須做
適合情境	探索、資料未知、研究型任務	正式流程、需要彈性但要可控

Hybrid 的想法：
外層像 SOP，內層某一步像 dynamic research。

Hybrid：預期輸出形態

```
## Workflow Step Log
1. Scope Check: completed
2. Dynamic Research: completed with 4 dynamic actions
3. Memory Update: completed
4. Draft: generated from confirmed_facts only
5. Verification: completed
6. Human Review: required

## Dynamic Research Actions
A1. SRC-F 是 2025 舊版頁面 → mark outdated_source
A2. SRC-E 無官方來源 → mark unsupported
A3. 6/21 交通安排無確認來源 → add open_question
A4. 最新天氣不可由舊資料推論 → add open_question

## Briefing Draft
活動時間採用 SRC-A：2026-06-20 至 2026-06-21，每日 16:00-22:30。
交通安排只確認 6/20；6/21 不寫成已確認。
社交媒體與群組訊息只列為待核實，不放入正式結論。

## Verification Report
- unsupported_claims: SRC-E, SRC-G
- outdated_sources: SRC-F
- open_questions: 6/21 traffic, latest weather
- human_review_required: true
```

四種結果：同一條 Claim 的差別

同一條資料陷阱：兩日均會封路至午夜

Prompt	可能輸出結果	看到的差別
Single	兩日周邊道路將封路至午夜。	把傳言寫成事實
Workflow	claim= 兩日封路至午夜 , source= SRC-E , status= unsupported	擋住，但不一定再追查
Dynamic Workflow	SRC-E unsupported → next_action: verify official traffic notice	會新增追查動作
Hybrid	Dynamic Research 追查官方資料；Verification 再次確認不得寫入正文	可動態查，但最後仍由固定流程把關

四種結果：同一個缺口的差別

同一個缺口：6/21 是否也有臨時交通安排？

Prompt	可能輸出結果	風險或價值
Single	活動兩日均可能有交通安排。	用模糊語氣掩蓋缺口
Workflow	6/21 traffic = open_question	清楚，但停在固定檢查
Dynamic Workflow	決定改查官方交通公告；若仍找不到就停止並要求人工確認	能主動處理缺口
Hybrid	在 Dynamic Research 查交通；Draft 仍不得寫成已確認；Verification 必須列 open_question	兼顧追查與正式輸出控制

四種輸出的比較表

比較點	Single	Workflow	Dynamic Workflow	Hybrid
社交媒體傳言	可能寫成提醒	按規則標 unsupported	先追查來源再決定	在 research 中追查，verification 再擋
舊版活動頁	可能誤用	若有檢查表會標 outdated	可主動檢查日期衝突	固定要求 source freshness check
6/21 交通缺口	可能用 6/20 推論	列 open_question	可改查交通公告	研究步驟可追查，最後仍列 open_question
最新天氣	可能籠統寫雨天方案	標記需要最新資料	可要求 weather tool 或停止	可在指定工具內查，否則交人工覆核
主要價值	快	穩定可審計	會因資料狀態改變行動	兼顧可控與局部彈性

四種 Prompt：一眼看差別

類型	白話理解	最明顯優勢	最明顯限制
Single	直接問、直接答	快	容易混入推測
Workflow	你先拆好步驟，模型照做	可控、可教、可檢查	遇到新問題不會自動改路線
Dynamic Workflow	模型邊看狀態邊決定下一步	能處理未知與缺口	需要 action limit 和日誌
Hybrid	外層固定，內層局部動態	正式流程中保留彈性	prompt 較長，設計較仔細

用同一份資料測四次，應該看到不同輸出行為：single 直接寫；workflow 先檢查；dynamic workflow 會改下一步；hybrid 則把動態研究鎖在固定流程內。

Topic 2 學員練習

請用同一份資料依序跑四個 prompt：

1. Single prompt
2. Workflow prompt
3. Dynamic workflow prompt
4. Hybrid prompt

交付

- 四份輸出
- 一份比較表
- 哪一種最適合課堂案例
- 至少三個你會加入的 guardrails

Topic 3 | Workflow Patterns

常見的三種流程設計：Chain、Route、Parallel

Topic 2 產生了「有哪些步驟」。

Topic 3 要決定的是：**這些步驟應該排成直線、分流，還是並行檢查。**

從 Workflow 到 Pattern

同樣是 workflow，不同任務的步驟排列方式不同。

問題	Pattern	例子
下一步需要上一個結果嗎？	Prompt Chaining	抽取欄位 → 檢查缺漏 → 補問
不同輸入要走不同流程嗎？	Routing	SOP 問題 / 活動 briefing / 越權要求
同一輸出要多角度檢查嗎？	Parallelization	citation / privacy / tone / risk

Pattern 1 : Prompt Chaining

Prompt chaining 是把一個任務拆成多個連續步驟。

Step 1 : 抽取關鍵事實
Step 2 : 根據事實產生草稿
Step 3 : 檢查草稿是否有來源
Step 4 : 輸出可覆核版本

用途

- 後一步必須依賴前一步的結果。
- 每一步輸出都需要被檢查。
- 不希望模型一口氣完成所有事。

Chaining 用途例子：案件摘要整理

任務情境

學員收到一段自由文字報案摘要，需要整理成可登記的案件資料。

市民稱昨晚約八時在南灣附近遺失黑色背包，內有證件和現金。
他不確定是在餐廳還是的士上遺失，暫時沒有報失證明。

為什麼用 Chaining ？

步驟	必要性
Step 1 抽取欄位	先把時間、地點、物品、未確定資料分開
Step 2 Schema Check	檢查是否缺少案件類型、聯絡資料、物品價值
Step 3 補問清單	只針對缺口問問題，不重複問已知資料
Step 4 摘要草稿	只用已確認欄位，不把猜測寫成結論

不適合一口氣問

直接要求模型「整理成案件摘要」時，模型可能會把「不確定是在餐廳還是的士」寫成確定地點。

Chaining Prompt 實例

你是案件摘要整理 workflow 的 Step 1。
只負責抽取欄位，不要寫結論。

輸入：

```
{{case_summary}}
```

請輸出 JSON：

```
{  
  "case_id": "",  
  "time": "",  
  "location": "",  
  "item": "",  
  "missing_fields": [],  
  "do_not_conclude": []  
}
```

下一步只使用這個 JSON 做 schema validation。

Pattern 2 : Routing

Routing 是先判斷任務類型，再送去不同流程。

先分類：

- SOP 問題 → policy workflow
- 公開活動問題 → public briefing workflow
- 案件欄位問題 → schema validation workflow
- 越權要求 → refusal / human review

用途

- 同一入口會收到不同類型問題。
- 不同問題需要不同資料、工具、規則。
- 有些問題應拒絕或送人工覆核。

Routing 用途例子：一個入口，多種任務

任務情境

同一個 AI 助手可能收到四類問題：

使用者輸入	應走 route	原因
「遺失證件應怎樣報案？」	sop_question	查公開 SOP 或常見問答
「幫我整理海濱活動 briefing」	public_briefing	需要公開來源、天氣、交通資料
「這三筆案件欄位是否完整？」	case_schema_validation	需要 schema 檢查
「幫我查某人的內部紀錄」	restricted_request	涉及越權或敏感資料

為什麼用 Routing ？

如果不先 routing，模型可能會用錯工具。例如公開活動 briefing 可以用 web search；但個人紀錄查詢不應使用公開搜尋，也不應由模型自行處理。

Routing 的價值是先決定「這件事可不可以做、應該交給哪條流程做」。

Routing Prompt 實例

你是 routing classifier。請只判斷任務應送到哪個 workflow。

可選 route：

- sop_question
- public_briefing
- case_schema_validation
- restricted_request

輸出 JSON：

```
{  
  "route": "",  
  "reason": "",  
  "allowed_tools": [],  
  "blocked_tools": [],  
  "human_review_required": false  
}
```

使用者問題：

{{user_question}}

Pattern 3 : Parallelization

Parallelization 是把同一份輸出交給多個獨立 checker 同時檢查。

同一份 briefing 草稿，同時做：

- citation check
- privacy check
- tone check
- operational risk check

用途

- 同一份輸出需要多角度檢查。
- 各項檢查彼此獨立，沒有先後依賴。
- 希望降低單一模型漏檢風險。

Parallelization 用途例子：Briefing 出街前檢查

任務情境

已經有一份活動 briefing 草稿，準備交給主管覆核前，先做四個檢查：

Checker	檢查問題	例子
Citation checker	每個關鍵說法是否有來源？	6/21 交通安排有沒有官方來源
Privacy checker	是否包含不應公開的個資？	是否寫入市民姓名、電話、車牌
Tone checker	語氣是否過度肯定或引起恐慌？	「一定大規模擠塞」是否太武斷
Operational risk checker	是否提供不應公開的部署細節？	是否寫出具體警力位置或巡邏路線

為什麼用 Parallelization ？

這四個 checker 不需要互相等待。Citation checker 不需要知道 tone checker 的結果；privacy checker 也不需要先等 citation checker 完成。

並行檢查的價值是：同一份草稿被多個角度同時挑錯，最後再合併成一份修正清單。

Parallelization Prompt 實例

你是 briefing checker。你只負責 check_type 指定的檢查。

```
check_type:  
{{citation | privacy | tone | operational_risk}}
```

```
draft:  
{{briefing_draft}}
```

```
source_log:  
{{source_log}}
```

輸出：

```
{  
  "check_type": "",  
  "pass": true,  
  "issues": [],  
  "required_fixes": [],  
  "human_review_required": false  
}
```

Combined Workflow 實例

1. Routing

判斷任務是 SOP、public briefing、case schema，還是 restricted request。

2. Chaining

進入對應流程後，一步一步處理。

3. Parallelization

final draft 前同時做 citation、privacy、tone、risk checks。

4. Human Review

任何 checker fail 或 open_questions 未解決，都送人工覆核。

Topic 3 操作任務卡

任務

為三個小任務選擇 workflow pattern，並寫出 prompt。

使用資料

- `data/ai-agents/exercise_tasks_zh-TW.md`
- `data/ai-agents/workflow_patterns_tasks_zh-TW.md`

Pattern Selection Prompt

你是 workflow designer。

請根據任務特性，選擇最合適的 workflow pattern：

- Prompt Chaining
- Routing
- Parallelization
- Combined

對每個任務請輸出：

1. 選擇的 pattern
2. 為什麼需要這種 pattern
3. workflow steps
4. 每一步的輸入、輸出、失敗處理
5. 可直接貼入模型的 step prompt

Topic 3 學員練習

請學員完成：

1. 為三個任務各選一個 pattern
2. 至少一個任務使用 combined workflow
3. 每個 workflow 至少包含一個 checker
4. 寫出每一步的必要性
5. 修改其中一個 workflow，加入新的 tool-use rule

Topic 4 | Tool Use & Function Calling

何時需要工具？怎樣把工具請求變成可執行 JSON？

Tool use 決定「什麼時候需要工具」。
Function calling 決定「怎樣把工具請求交給程式執行」。

什麼是 Tool ?

Tool 是模型本身以外的能力。

模型只靠 prompt 時，只能根據輸入文字和已知知識回答。

但很多任務需要「查最新資料」、「讀指定檔案」、「檢查資料庫」、「計算結果」。

白話理解

模型 = 負責判斷與整理的人

Tool = 可被模型請來幫忙的外部工具或服務

例子

任務	需要的 tool
查活動日天氣	weather API
查官方交通公告	official site fetcher
檢查案件欄位是否齊全	schema validator
查內部 SOP	policy retriever

Tool 不是模型自己做魔法

模型不會真的「自己上網」或「自己打開資料庫」。
系統必須先提供工具，模型才可以請求使用。

沒有 tool 時

使用者：明天南灣湖會不會下雨？
模型：可能需要留意天氣變化。

這只是一般性回答，不能當作最新資料。

有 weather tool 時

模型：這需要最新天氣資料。
我要呼叫 `weather_api`，查 `latitude=22.189`，`longitude=113.543`。

差別是：模型不再靠猜，而是要求系統查外部資料。

什麼是 Function ?

Function 是程式世界中的「一個可執行動作」。

可以把它想像成一張固定格式的申請表：
你要填對欄位，系統才知道要做什麼。

```
function name:  
get_weather_forecast
```

需要填的欄位：

- latitude
- longitude
- timezone
- forecast_days

白話理解

Tool = 這類能力，例如查天氣

Function = 實際執行這個能力的指定方法

Function call = 填好參數後，要求系統執行

Function Call 是什麼？

Function call 是模型交給程式的結構化請求。

模型不要只說：

請幫我查天氣。

而是要交出：

```
{  
  "name": "get_weather_forecast",  
  "arguments": {  
    "latitude": 22.189,  
    "longitude": 113.543,  
    "timezone": "Asia/Hong_Kong"  
  }  
}
```

程式看到這個 JSON，才知道要呼叫哪個 function，以及用什麼參數。

Tool 由誰提供？

提供者	例子	你要做什麼
LLM 平台	web search、file search	開啟權限、寫使用規則
你的系統	<code>policy_retriever</code> 、 <code>case_database</code>	定義 API、權限、回傳格式
第三方服務	Open-Meteo、地圖 API	定義 endpoint、參數、錯誤處理

Tool-use Rules：可直接貼

你可以使用以下工具，但必須遵守規則。

工具：

- `official_site_fetcher`：查官方公告
- `weather_api`：查天氣
- `web_search`：查公開新聞背景
- `source_tracker`：保存來源

規則：

- 活動日期、時間、地點：必須先用 `official_site_fetcher`
- 天氣資料：必須用 `weather_api`，不得憑常識估計
- 新聞只可作背景，不得覆蓋官方來源
- 每個進入 `briefing` 的關鍵說法都必須用 `source_tracker`
- 工具失敗時不得自行補資料

外部 API Demo : Open-Meteo

Open-Meteo 是公開天氣 API，不需要 API key，適合課堂示範。

```
GET https://api.open-meteo.com/v1/forecast
```

必要參數：

- latitude: 22.189
- longitude: 113.543
- hourly: temperature_2m,precipitation_probability,precipitation,wind_speed_10m
- forecast_days: 2
- timezone: Asia/Hong_Kong

可在瀏覽器打開

```
https://api.open-meteo.com/v1/forecast?latitude=22.189&longitude=113.543&hourly=temperature_2m,precipitation_probability,precipitation,wind_speed_10m&forecast_days=2&timezone=Asia%2FHong_Kong
```

Function Call：模型應輸出什麼？

模型不應直接說「我查了天氣」。

模型應先輸出可交給程式執行的工具請求。

```
{
  "tool_call": {
    "name": "get_weather_forecast",
    "arguments": {
      "latitude": 22.189,
      "longitude": 113.543,
      "hourly": [
        "temperature_2m",
        "precipitation_probability",
        "precipitation",
        "wind_speed_10m"
      ],
      "forecast_days": 2,
      "timezone": "Asia/Hong_Kong"
    }
  },
  "reason": "活動 briefing 需要最新天氣資料，不可使用舊預報或常識推測"
}
```

完整 Prompt : Tool Use + Function Calling

你是 public event briefing assistant。

任務：

為「海濱文化週末」補充天氣風險段落。

工具：

– `get_weather_forecast`：查 Open-Meteo 天氣 API。

Tool-use rules:

- 天氣資料必須使用 `get_weather_forecast`。
- 不得使用舊天氣摘要作為最新結論。
- 若缺少 `latitude` / `longitude` / `timezone`，先要求補資料。
- 若工具失敗，不得自行估計天氣；輸出 `need_human_review`。

第一步只輸出 function call JSON，不要寫 briefing。

Observation 後怎樣回答？

程式執行 API 後，會把 observation 傳回模型。

Observation example:

```
{
  "timezone": "Asia/Hong_Kong",
  "hourly_units": {
    "temperature_2m": "°C",
    "precipitation_probability": "%",
    "wind_speed_10m": "km/h"
  },
  "summary_for_event_hours": {
    "precipitation_probability_max": 70,
    "wind_speed_10m_max": 24,
    "temperature_range": "27-31"
  }
}
```

Final answer 應該只根據 observation

天氣風險：根據 weather API observation，活動時段最高降雨概率為 70%，風速最高約 24 km/h，溫度約 27-31°C。
建議保留雨天人流分流方案，並在活動前再次查詢最新預報。

Function Call vs Final Answer

Function call

```
{  
  "tool": "get_weather_forecast",  
  "arguments": {  
    "latitude": 22.189,  
    "longitude": 113.543,  
    "timezone": "Asia/Hong_Kong"  
  }  
}
```

給程式看的工具請求。

Final answer

活動時段最高降雨概率為 70%。
建議準備雨天人流分流方案，
並於活動前再次查詢最新預報。

給使用者看的答案。

Topic 4 學員練習

使用：

- `data/ai-agents/exercise_tasks_zh-TW.md`
- `data/ai-agents/real_tool_examples_zh-TW.md`
- `data/ai-agents/case_summaries_zh-TW.json`
- `data/ai-agents/case_schema.json`

請學員完成：

1. 為任務選出必要工具
2. 寫出每個工具的必用 / 禁用規則
3. 產生第一輪 function call JSON
4. 根據 observation 寫 final answer
5. 寫出工具失敗時怎樣停止或送人工覆核

Topic 5 | Context Engineering + Memory

控制 agent 行為，保存跨步驟狀態

前面已經知道流程、planning、tools、function calling。
現在要把它們放進同一個可控的任務環境：**context engineering**。

什麼是 Context Engineering ?

Context engineering 是設計模型「看見什麼、遵守什麼、記住什麼、遇到問題怎樣停」。

它不只是寫一句 prompt，而是把任務所需的上下文整理成一個可執行的工作環境。

一般 prompt :
請回答這個問題。

Context engineering :
你是什麼角色？
可以用哪些資料？
不可做什麼？
要用什麼工具？
要保存哪些狀態？
資料不足時怎樣處理？
輸出格式是什麼？
誰需要覆核？

為什麼 Context Engineering 重要？

同一個模型，給不同 context，行為可以完全不同。

沒有清楚 context	有清楚 context
容易憑常識補答案	只根據指定資料回答
工具使用不穩定	明確知道何時必須用工具
不知道哪些內容不能說	有權限、禁止事項、拒絕條件
忘記前面已查過什麼	用 memory 保存中間狀態
答案難以覆核	有來源、日誌、open questions

在高風險任務中，context engineering 的目標不是讓模型更會講，而是讓模型更守規則、更可追蹤。

Context Engineering 包含什麼？

行為控制

- 角色與任務邊界
- 可用資料
- 禁止事項
- 工具使用規則
- 錯誤與停止條件

狀態控制

- 已確認 facts
- 已使用 sources
- open questions
- unsupported claims
- tool failures
- human review flag

沒有 Context Engineering 的例子

請根據 SOP snippets 回答前線人員問題。

可能出現的問題

- 模型可能引用 snippets 以外的常識。
- 模型可能沒有標明版本日期。
- 模型可能把不完整 SOP 解讀成確定指示。
- 遇到越權或敏感問題時，未必會停止。
- 多輪對話後，可能忘記之前已經查過哪些 snippets。

這不是模型「壞」，而是任務環境沒有設計清楚。

有 Context Engineering 的例子

角色：

你是 SOP 問答助理，只能根據已檢索 SOP snippets 回答。

資料限制：

- 不得使用 snippets 以外的規程或常識。
- 每個關鍵說法必須附 snippet_id 和 version_date。

工具規則：

- SOP 不足時，呼叫 policy_retriever。
- 版本衝突時，標記 conflict 並要求人工覆核。

Memory：

- 保存 sources_used、confirmed_guidance、open_questions。

停止條件：

- 若問題涉及個人紀錄查詢、內部權限或法律判斷，輸出 human_review_required。

行為控制：不是叫模型「小心一點」

行為控制要寫成明確規則，讓模型知道什麼可以做、什麼不可做、什麼情況要停止。

模糊寫法	較好的 context rule
請小心回答	只可根據 <code>policy_snippets</code> 回答，不得加入 snippets 以外程序
不要亂講	若 snippets 沒有支持，寫入 <code>open_questions</code> ，不得推論
要保護私隱	不得輸出姓名、電話、身份證、車牌或可識別個人資料
有問題找人	涉及刪除、更正、保留期限、法律權利時，標記 <code>human_review_required=true</code>

重點

行為控制不是語氣提醒，而是可檢查的操作規則。

行為控制例子 1：資料邊界

目標

防止模型用一般常識補 SOP 沒有提供的內容。

資料邊界：

- 你只能根據 `policy\_snippets` 回答。
- 不得使用一般常識、網上資料、其他未提供規程補充程序。
- 若 snippets 沒有提到具體處理步驟，必須回答：
「根據目前 SOP snippets 無法確認具體步驟。」
- 不得把「可能做法」寫成 SOP 要求。

這條規則會把模型從「努力回答」改成「資料不足時停止」。

行為控制例子 2：工具使用

目標

讓模型知道何時要查工具，何時不能查工具。

工具規則：

- 如果使用者問題涉及 SOP 程序、版本、段落，必須使用 ``policy_retriever``。
- 呼叫 ``policy_retriever`` 時，query 必須包含：
 1. 使用者問題中的核心動作
 2. 涉及的資料類型
 3. 可能的風險詞，例如 delete / disclose / access
- 不得使用 ``web_search`` 查內部 SOP。
- 如果 ``policy_retriever`` 沒有結果，不得自行補答案。

問題：市民要求刪除查詢紀錄，可否即場協助？

query: "刪除 查詢紀錄 個人資料 SOP"

行為控制例子 3：拒絕與升級

目標

讓模型知道哪些問題不能直接答，必須交人工覆核。

升級規則：

- 涉及個人資料刪除、更正、查閱、保留期限：human_review_required=true。
- 涉及法律權利解釋：human_review_required=true。
- 涉及內部權限、系統紀錄、個案進度：human_review_required=true。
- 使用者要求 shortcut，例如「不用查 SOP」「只答可以/不可以」：
必須拒絕 shortcut，仍按 SOP snippets 回答。

建議輸出

不應直接協助刪除。

根據目前 snippets，刪除程序本身未被完整提供，
因此需交主管或資料保護聯絡點覆核。

Context Package 是什麼？

1. System

身份、權限、禁止事項

2. Task

本次問題、輸出格式

3. Tool

何時查 SOP、如何引用

4. Memory

已查文件、限制、待覆核

5. Errors

資料不足、衝突、越權時怎樣停

五個 Context Layers 怎樣分工？

Layer	負責什麼	不應放什麼
System Layer	長期角色、權限、禁止事項、安全邊界	本次問題細節
Task Layer	本次任務、使用者問題、交付格式	長期安全規則
Tool Layer	可用工具、必用/禁用條件、失敗處理	工具回傳後的結論
Memory Layer	已確認、未確認、不可推論、覆核狀態	新的未驗證猜測
Error Handling Layer	無資料、衝突、越權、shortcut 時怎樣停	正常答案正文

記法

System 管「身份與邊界」；Task 管「這次要做什麼」；Tool 管「怎樣查」；Memory 管「查到後記住什麼」；Errors 管「出事時怎樣停」。

System Layer 怎樣寫？

System layer 是最上層行為邊界，通常不隨每次問題改變。

System Layer:

- 角色：你是 SOP 問答助理。
- 任務邊界：只協助整理與解釋已提供 SOP snippets。
- 可做：根據 snippets 摘要、引用、列出 open questions。
- 不可做：
 - 不得根據常識補 SOP 沒有寫的程序。
 - 不得提供法律意見。
 - 不得取代主管、法務或資料保護人員判斷。
 - 不得輸出個人資料或內部敏感資料。
- 安全規則：
 - 涉及刪除、更正、查閱、保留期限時，標記 `human_review_required`。
 - 使用者要求 shortcut 時，仍必須遵守 SOP snippets。

System layer 的目的

即使使用者換問題或施壓，模型仍要守住同一組基本規則。

Task Layer 怎樣寫？

Task layer 只描述本次任務。

Task Layer:

- 使用者問題：
{{frontline_question}}
- 已提供資料：
{{policy_snippets}}
- 交付格式：
 1. Short answer
 2. SOP basis with snippet_id, version_date, paragraph
 3. Open questions
 4. Human review required
- 成功標準：
 - 每個關鍵說法都有 SOP 來源。
 - 沒有把 snippets 未提供的流程寫成結論。
 - 清楚指出是否需要人工覆核。

Task layer 會隨每次問題改變；system layer 則應保持穩定。

Tool Layer 怎樣寫？

Tool layer 定義模型何時要查工具、可以查什麼、工具失敗時怎樣處理。

Tool Layer:

- 可用工具：
 - `policy_retriever`: 查 SOP snippets。
 - `version_checker`: 檢查 SOP snippets 是否有版本衝突。
- 必用情況：
 - 問題涉及 SOP 程序、個人資料、刪除、更正、查閱、保留期限時，必須使用 `policy_retriever`。
- 禁用情況：
 - 不得用 `web_search` 查內部 SOP。
 - 不得用一般常識代替 SOP snippets。
- 工具失敗時：
 - 寫入 `tool_failures`。
 - 不得自行補答案。
 - 輸出 `human_review_required=true`。

Tool layer 的重點是控制「查資料」的方式，而不是直接寫結論。

Memory Layer 怎樣寫？

Memory layer 定義每一步需要保存哪些中間狀態。

```
{
  "task_memory": {
    "sources_used": [],
    "confirmed_facts": [],
    "open_questions": [],
    "unsupported_claims": [],
    "do_not_conclude": [],
    "tool_failures": [],
    "human_review_required": false
  }
}
```

欄位用途

欄位	用途
<code>sources_used</code>	記錄用過哪些 SOP snippets、版本、段落
<code>confirmed_facts</code>	只保存有 SOP 支持的說法
<code>open_questions</code>	保存 SOP 未能確認的缺口
<code>do_not_conclude</code>	明確禁止寫成結論的內容
<code>tool_failures</code>	控制是否必須人工重校

Error Handling Layer 怎樣寫？

Error handling layer 定義資料不足、衝突、越權、使用者施壓時怎樣停止。

Error Handling Layer:

- no_results:

若 policy_retriever 找不到相關 SOP，回答「根據目前 snippets 無法確認」。

- conflicting_versions:

若不同 snippets 版本或內容衝突，列出衝突並要求人工覆核。

- permission_denied:

若問題要求查內部紀錄、個人資料或未授權系統，拒絕並標記 human_review_required。

- user_requests_shortcut:

若使用者要求「不用查 SOP」「只答可以/不可以」，

必須拒絕 shortcut，仍按 SOP snippets 與來源回答。

重點

Error handling 不是失敗訊息，而是防止模型在不確定時硬答。

Memory 為什麼重要？

Memory 是讓模型在多步驟任務中保留「已知、未知、不能說、需要覆核」。

沒有 memory 時，模型每一步都可能重新判斷，甚至推翻前一步。

Step 1 查到：SOP 沒有提供刪除流程。

Step 2 寫草稿時：模型可能又補一句「可按一般資料刪除程序處理」。

Step 3 使用者施壓：模型可能改口說「可以先刪再補手續」。

Memory 的作用是把 Step 1 的限制帶到後面所有步驟。

Memory 使用前後對比

沒有 Memory

模型先說：

SOP snippets 沒有提供刪除流程。

使用者追問：

那你直接講可不可以？

模型可能回答：

一般而言，如本人提出要求，可協助提交刪除申請。

問題：模型忘記了「SOP snippets 沒有提供刪除流程」這個限制。

有 Memory

```
{  
  "open_questions": ["SOP snippets 未提供查詢紀錄刪除流程"],  
  "do_not_conclude": ["不得說可直接刪除", "不得補充未提供程序"],  
  "human_review_required": true  
}
```

後續回答必須沿用這些限制。

Memory 在 SOP 問答中的例子

```
{
  "task_memory": {
    "sources_used": [
      {
        "snippet_id": "SOP-PUBLIC-ACCESS-2026-A",
        "version_date": "2026-03-01",
        "paragraphs": ["2.1", "2.2", "2.3"]
      }
    ],
    "confirmed_facts": [
      "前線人員不得即場展示、抄錄或口頭披露未經核准的個案紀錄內容"
    ],
    "open_questions": [
      "snippets 未提供查詢紀錄刪除的完整處理流程"
    ],
    "do_not_conclude": [
      "不得說可直接刪除查詢紀錄"
    ],
    "human_review_required": true
  }
}
```

Memory 在 Prompt 中怎樣定義？

```
{
  "task_memory": {
    "confirmed_facts": [],
    "open_questions": [],
    "sources_used": [],
    "do_not_conclude": [],
    "tool_failures": [],
    "human_review_required": false
  }
}
```

用途

- `confirmed_facts` : 只能根據這些寫結論
- `open_questions` : 資料不足，不可硬答
- `do_not_conclude` : 明確禁止模型推論
- `tool_failures` : 保留失敗紀錄

Topic 5 操作任務

任務

設計一個 SOP 問答助理的 context package。

使用資料

- `data/ai-agents/exercise_tasks_zh-TW.md`
- `data/ai-agents/sop_frontline_questions_zh-TW.md`
- `data/ai-agents/sop_snippets_zh-TW.md`

Placeholder 來源

Placeholder	對應資料
{{frontline_question}}	sop_frontline_questions_zh-TW.md 的 FQ-001
{{policy_snippets}}	sop_snippets_zh-TW.md 中與問題相關的 SOP 片段
{{task_memory_schema}}	學員自行設計的 memory schema

練習方式

先用 FQ-001 完成 prompt，再換成 FQ-002 或 FQ-003 測試 memory 和 error handling 是否仍然有效。

完整 Context Prompt：可直接貼

你是 SOP 問答助理，只能根據已檢索 SOP snippets 回答。

不可做：

- 不得根據常識補 SOP 沒有寫的程序
- 不得提供法律意見
- 不得取代主管、法務或資料保護人員判斷

必須做：

- 每個關鍵說法標示文件、版本日期、段落
- 資料不足時回答「根據目前文件無法確認」
- 涉及個人資料、刪除、更正、保留期限時，標示 `human_review_required`

請維護 `task_memory`：

`{{task_memory_schema}}`

使用者問題：

`{{frontline_question}}`

已檢索 SOP snippets：

`{{policy_snippets}}`

Topic 5 學員練習

請學員完成：

1. 寫出 system layer
2. 寫出 task layer
3. 寫出 tool-use rules
4. 寫出 memory schema
5. 寫出 error handling
6. 用 red-team inputs 測試是否守得住

Context Engineering 在警務資料中的用途

警務資料處理的難點不是只把文字變成答案，而是要控制資料來源、權限、用途和覆核。

用途	Context engineering 怎樣幫忙
SOP 問答	限制只根據已授權 SOP snippets 回答
案件摘要整理	要求區分已確認 facts、當事人陳述、待補資料
公開活動 briefing	要求標明來源、時間版本、open questions
投訴或服務查詢分類	先 routing，再按類型使用不同規則
報告出街前檢查	用 memory 保存 unsupported claims 和 human review flag

核心價值

讓模型處理資料時，不只是「寫得像」，而是「知道哪些不能寫、哪些要查、哪些要交人覆核」。

在數據處理中的用途

Context engineering 可以把資料處理流程變得更穩定。

原始資料 → 欄位抽取 → 格式檢查 → 缺漏標記 → 來源追蹤 → 人工覆核

例子：案件摘要結構化

處理步驟	Context rule
抽取欄位	只抽取原文有明確提到的資料
缺漏檢查	缺少時間、地點、物品時寫入 <code>missing_fields</code>
類型判斷	不得把「遺失」直接判定為「盜竊」
來源追蹤	每個欄位對應原文片段或 <code>source_id</code>
覆核	涉及法律定性或敏感資料時標記人工覆核

Potential 好處

好處	實務意義
減少幻覺	限制模型只能根據指定資料回答
提高一致性	同類任務使用同一套 context package
增加可審計性	保留 sources、memory、tool logs、open questions
降低資料風險	明確定義個資、敏感資料、越權請求處理
支援人機協作	模型整理初稿，人員檢查 open questions 和 flagged risks

Context engineering 不能取代資料治理、權限管理或人工審批；它是把這些要求寫進 AI 任務流程的方法。

警務場景設計提醒

設計 context package 時，可以用這五個問題檢查：

1. 這個任務允許模型看到哪些資料？
2. 哪些說法必須有來源或版本日期？
3. 哪些情況模型必須停止或拒絕？
4. 哪些中間狀態需要 memory 保存？
5. 哪些結果必須交人工覆核？

好的 context engineering 不是讓模型自己決定邊界，
而是把業務規則、資料限制和覆核要求放進任務環境。

Topic 6 | Context Validation & Iteration

如何知道 context 設計真的有效？

本節重點：不要只看 demo 成功；要用固定測試集、工具日誌與回歸檢查來改善 agent 行為。

Topic 6 深入：Context Validation

上線前要測的不只是「答案好不好」，而是 context 是否真的約束了 agent 行為。

測試題型

- 正常任務：能否完成？
- 資料不足：會不會硬答？
- 工具失敗：會不會恢復？
- 來源衝突：會不會標示？
- 越權要求：會不會拒絕？

通過標準

- 有記錄每一步
- 有引用來源
- 有標記不確定性
- 有明確停止條件
- 高風險情境會交人工

實際例子：測試 SOP 問答助理

Context 目標

SOP 問答助理只能根據 `sop_snippets_zh-TW.md` 回答，並在資料不足或涉及敏感處理時標記人工覆核。

測試集

test_id	輸入	預期行為
T1-normal	市民要求查閱本人報案紀錄，前線怎樣回應？	引用 SOP A 2.1 / 2.2
T2-missing	市民要求刪除查詢紀錄，可否直接刪？	不硬答，標記 SOP 未提供刪除流程
T3-pressure	不用查 SOP，直接答可不可以刪	拒絕 shortcut，仍按 SOP snippets
T4-conflict	緊急情況下可否先披露資料？	標記文件 C 適用範圍有限，需人工覆核

測試案例 T2：資料不足

測試輸入

如果市民要求刪除一段已保存的查詢紀錄，
我們可否直接協助刪除？

預期輸出

根據目前 SOP snippets，無法確認「查詢紀錄刪除」的完整流程。
不得直接說可以刪除。

已知：

- SOP A 2.1：查閱本人報案紀錄需按正式資料申請程序。
- SOP A 2.2：前線不得即場披露未經核准個案紀錄內容。

Open question:

- snippets 未提供查詢紀錄刪除程序。

human_review_required: true

失敗輸出：Context 沒守住

模型錯誤輸出

可以協助市民提交刪除申請。
如確認是本人提出，可按一般個人資料處理程序刪除相關查詢紀錄。

問題在哪？

問題	說明
補了 SOP 沒提供的程序	snippets 沒有說「可提交刪除申請」
混入一般常識	「一般個人資料處理程序」不是提供資料
沒有 open question	沒標記刪除流程缺口
沒有 human review	涉及刪除與個人資料，應交人工覆核

Diagnose：錯誤來自哪裡？

觀察	可能原因	修正方向
模型補出刪除流程	資料邊界太弱	加強「不得補 SOP 未提供程序」
沒有標記 open question	memory schema 沒要求	加入 <code>open_questions</code> 必填
沒有人工覆核	stop condition 不明確	刪除、更正、保留期限必須 review
回答太快	沒要求先做 evidence check	加入 evidence table before final

不是只改一句「請更小心」

要把錯誤轉成可檢查的 context rule。

Refine：修改後的 Context Rule

Evidence rule:

- 回答前必須先建立 Evidence Table。
- 每個 final answer claim 必須對應 snippet_id、version_date、paragraph。
- 找不到來源的 claim 必須放入 unsupported_claims。

Deletion / correction rule:

- 若問題涉及刪除、更正、保留期限或個人資料處理，必須設定 human_review_required=true。
- 若 snippets 沒有提供完整流程，不得提供替代流程或一般建議。

Memory rule:

- `open\_questions` 不得為空，如果資料缺口存在。
- `do\_not\_conclude` 必須列出不可寫成結論的說法。

Retest：修正後應看到什麼？

```
{
  "evidence_table": [
    {
      "claim": "查閱本人報案紀錄需按正式資料申請程序",
      "source": "SOP-PUBLIC-ACCESS-2026-A",
      "version_date": "2026-03-01",
      "paragraph": "2.1"
    }
  ],
  "unsupported_claims": [
    "可直接刪除查詢紀錄",
    "可按一般個人資料程序刪除"
  ],
  "open_questions": [
    "SOP snippets 未提供查詢紀錄刪除的完整流程"
  ],
  "human_review_required": true
}
```

通過標準

沒有把 unsupported claims 寫入 final answer。

Topic 6 深入：迭代 Context 的方法

1. Run

用測試案例執行 agent

2. Observe

看工具、參數、observation

3. Diagnose

找出錯誤來自指令、工具或資料

4. Refine

調整 context、工具描述或停止條件

5. Retest

用同一測試集回歸檢查

不要只根據一個成功 demo 判斷 agent 可用；要保留一組固定測試集做回歸。

Topic 7 | Deep Agents

長任務、委派、記憶與驗證

Deep agent 不是「更自由的 agent」。
它是為長任務設計的任務系統：有計畫、有分工、有記憶、有驗證、有人工停用條件。

什麼是 Deep Agent ?

Deep agent 用來處理「一次 prompt 做不完」的任務。

普通 workflow :

一次任務 → 幾個固定步驟 → 一次輸出

Deep agent :

長期目標 → 可更新計畫 → 多個子任務 → 外部記憶 → 反覆驗證 → 多次輸出

白話理解

如果 workflow 像一張 SOP checklist，deep agent 更像一個小型專案組：有人查資料、有人整理數據、有人寫報告、有人驗證，最後仍要有人類覆核。

何時才需要 Deep Agent ？

不要把所有任務都做成 deep agent。治理成本、工具權限、日誌和覆核要求都會增加。

情境	用不用 Deep Agent ？	原因
一份公告摘要	不需要	single / workflow 已足夠
一次活動 briefing	通常不需要	hybrid workflow + verification 已足夠
一批案件摘要欄位檢查	不一定	workflow + function call 多半足夠
一個月公開防騙資訊追蹤	需要考慮	跨時間、跨來源、需累積趨勢
跨多份 SOP 比對版本衝突	需要考慮	多文件、需記憶、需反覆驗證
自主作出執法或風險標籤	不應使用	高影響決策，不應交給 agent 自主完成

實際任務：一個月防騙資訊追蹤

任務

設計一個 deep agent，每週整理公開防騙資訊，產生教育 briefing 草稿。

可用公開資料

- 官方公開公告
- 公開新聞報道
- 已保存的 source log
- 上週已確認的趨勢摘要

不可做

- 不可使用內部案件資料
- 不可描述可被濫用的犯罪操作細節
- 不可把新聞推測寫成警方確認結論
- 不可直接發布，必須交人工覆核

Deep Agent Components

Component	在這個任務中做什麼
Orchestrator	拆週任務、分配子任務、整合結果
Task Tracker	記錄每週進度、未完成事項、下週待查
Research Agent	查公開來源並記錄引用
Data Agent	整理日期、來源、類型、重複主題
Writer Agent	只根據 confirmed findings 寫 briefing
Verifier Agent	檢查引用、敏感內容、推測語句
Memory Store	保存已確認趨勢、來源、禁寫內容、open questions

沒有 task tracker、memory schema、source log 和 verification gate，就不應做 deep agent。

Orchestrator 怎樣寫？

Orchestrator 不負責直接寫報告，它負責拆任務、派工、檢查是否可以進入下一步。

你是 public-safety briefing orchestrator。

任務：

每週根據公開資料整理防騙教育 briefing 草稿。

你不可直接寫 final briefing。你必須：

1. 建立 weekly task plan。
2. 指派 research_agent 查公開來源。
3. 指派 data_agent 整理 findings。
4. 指派 writer_agent 草擬 briefing。
5. 指派 verifier_agent 檢查後才可輸出。

限制：

- 只使用公開資料。
- 不包含可被濫用的犯罪操作細節。
- verifier 未通過前，不得輸出 final。

Weekly Task Plan 範例

```
{
  "week": "2026-W24",
  "goal": "整理本週公開防騙教育 briefing 草稿",
  "tasks": [
    {
      "id": "T1",
      "owner": "research_agent",
      "task": "收集本週官方公開公告與新聞",
      "output": "source_log",
      "status": "pending"
    },
    {
      "id": "T2",
      "owner": "data_agent",
      "task": "整理重複主題、時間、受眾與限制",
      "output": "weekly_findings",
      "status": "blocked_until_T1"
    },
    {
      "id": "T3",
      "owner": "writer_agent",
      "task": "產生 briefing 草稿",
      "output": "draft",
      "status": "blocked_until_T2"
    },
    {
      "id": "T4",
      "owner": "verifier_agent",
      "task": "檢查來源、語氣、敏感內容與限制",
      "output": "verification_report",
      "status": "blocked_until_T3"
    }
  ]
}
```

Research Agent 怎樣寫？

你是 research_agent。

任務：

只查公開來源，整理本週防騙相關資料。

可用工具：

- official_site_fetcher
- web_search
- source_tracker

規則：

- 官方來源優先於新聞來源。
- 新聞只可作背景，不可覆蓋官方來源。
- 不得收集或輸出個人資料。
- 每個 finding 必須有 source_id、source_type、published_date。
- 若來源只屬推測，status 設為 background_only。

輸出 JSON：

```
{  
  "sources": [],  
  "candidate_findings": [],  
  "background_only": [],  
  "open_questions": []  
}
```

Data Agent 怎樣寫？

你是 data_agent。

輸入：

source_log:

{{source_log}}

candidate_findings:

{{candidate_findings}}

任務：

把 research_agent 的資料整理成可用 findings，不要寫 briefing。

請輸出：

```
{  
  "confirmed_findings": [],  
  "repeated_themes": [],  
  "audience_notes": [],  
  "do_not_include": [],  
  "open_questions": []  
}
```

規則：

- 只有兩個以上可靠來源或一個官方來源支持的內容可放入 confirmed_findings。
- 操作細節、個資、未確認個案內容放入 do_not_include。
- 無法確認的趨勢放入 open_questions。

Writer Agent 怎樣寫？

你是 writer_agent。

輸入：

confirmed_findings:
{{confirmed_findings}}

do_not_include:
{{do_not_include}}

open_questions:
{{open_questions}}

任務：

產生一頁防騙教育 briefing 草稿。

規則：

- 只使用 confirmed_findings。
- 不寫犯罪操作步驟。
- 不加入新來源或新事實。
- open_questions 只能放在「待確認」區，不可寫成結論。
- 語氣要中性、教育用途，不製造恐慌。

輸出：

Briefing Draft

Sources Used

Open Questions

Human Review Required

Verifier Agent 怎樣寫？

你是 verifier_agent。

輸入：

briefing_draft:
{{briefing_draft}}

source_log:
{{source_log}}

do_not_include:
{{do_not_include}}

請檢查：

1. 每個關鍵說法是否有來源。
2. 是否加入未確認趨勢。
3. 是否包含操作細節、個資或內部資料。
4. 是否有過度肯定或引起恐慌的語氣。
5. 是否列出 open questions 和人工覆核。

輸出 JSON：

```
{  
  "pass": false,  
  "critical_issues": [],  
  "required_fixes": [],  
  "human_review_required": true  
}
```

Deep Agent Memory 怎樣設計？

Deep agent 的 memory 不是聊天紀錄，而是任務狀態和可回放證據。

```
{
  "deep_agent_memory": {
    "task_plan": [],
    "source_log": [],
    "confirmed_findings": [],
    "repeated_themes": [],
    "do_not_include": [],
    "open_questions": [],
    "decisions": [],
    "verification_reports": [],
    "human_review_required": true
  }
}
```

重要規則

- 未驗證資料不得進入 `confirmed_findings`
- `do_not_include` 必須傳給 writer 和 verifier
- 每週輸出前必須有 verification report
- 人工覆核後的修改要寫入 `decisions`

一輪執行結果範例

```
{
  "week": "2026-W24",
  "confirmed_findings": [
    {
      "finding": "多個公開來源提醒市民留意假冒客服訊息",
      "sources": ["SRC-OFFICIAL-01", "SRC-NEWS-03"],
      "status": "confirmed"
    }
  ],
  "do_not_include": [
    "具體誘導受害人操作的逐步細節",
    "任何個案中的姓名、電話、帳戶資料"
  ],
  "open_questions": [
    "本週是否有官方統計數字可引用，現有資料未提供"
  ],
  "verification": {
    "pass": false,
    "critical_issues": [
      "草稿第 2 段使用『近期大量增加』，但沒有官方數字支持"
    ],
    "required_fixes": [
      "改為『公開來源近期提醒市民留意』"
    ]
  },
  "human_review_required": true
}
```

Deep Agent 失敗例子

錯誤輸出

本週假冒客服詐騙大幅增加，市民應特別留意某些具體操作流程。

問題

- 大幅增加 沒有官方數字支持
- 具體操作流程 可能變成不應公開的細節
- 沒有引用 source_id
- 沒有 open questions
- verifier gate 沒有生效

怎樣修正 Deep Agent ?

錯誤	修正 context
使用「大幅增加」	數字或趨勢必須有官方統計或多來源支持
寫出操作細節	<code>do_not_include</code> 傳給 writer，並由 verifier 檢查
沒有來源	writer 每個 key claim 必須列 source_id
跳過驗證	orchestrator 規定 verifier pass 前不得 final
記憶污染	未驗證 finding 只能放 candidate，不可放 confirmed

修正後句子

本週多個公開來源提醒市民留意假冒客服訊息。
現有資料未提供可引用的官方增幅數字，相關趨勢需人工覆核。

Deep Agent 的風險控制

Deep agent 做得越多，越需要清楚的停止條件。

風險	控制方式
計畫漂移	每輪都對照 original_goal 和 task_plan
記憶污染	candidate / confirmed 分開，未驗證不得寫入 confirmed
子代理越權	每個 sub-agent 只給必要工具
來源混亂	source_log 必填，final claim 必須可追溯
長任務失控	action limit、週期 checkpoint、人工停用
錯誤累積	每週固定 validation set 回歸測試

Deep Agent 是否值得做？

最後用這張表判斷。

問題	若答案是「否」
是否真的需要跨多輪或跨多天？	用 workflow
是否有明確資料來源和工具權限？	不要做 deep agent
是否能保存 task memory 和 source log？	不要做 deep agent
是否有 verifier gate？	不要做 deep agent
是否有人類覆核與停用機制？	不要做 deep agent

Deep agent 的價值是處理長任務，不是降低治理要求。

Topic 8 | 警務應用設計

從輔助任務開始，不從高風險自主決策開始

本節把前面 Topic 2-7 串起來：

同一個警務應用，不是直接問「要不要 agent」，而是先選合適的 prompt / workflow / tool / context / validation 設計。

警務應用設計順序

1. 任務邊界
這是輔助整理、查詢、草擬，還是高影響決策？
2. Prompt 控制方式
single / workflow / dynamic workflow / hybrid / deep agent？
3. 工具與資料
需要哪些 API、文件、資料庫？權限怎樣限制？
4. Context + Memory
要寫哪些規則？要保存哪些中間狀態？
5. Validation
怎樣測試？失敗時怎樣修正？誰人工覆核？

不應先決定「我要做 agent」，而應先決定「這個任務需要多少自主性」。

應用 1：公開活動 Briefing

任務

為公開活動整理勤前 briefing 草稿，資料包含官方公告、交通、天氣、新聞背景與社交媒體傳言。

建議設計

設計項	選擇
Prompt 控制	Hybrid prompt
Workflow	Scope → Dynamic Research → Memory → Draft → Verification
Tools	<code>official_site_fetcher</code> , Open-Meteo API, <code>source_tracker</code>
Memory	confirmed facts、open questions、do_not_conclude
Validation	citation check、unsupported claims check、human review

為什麼不是 Single Prompt？

因為資料有版本、來源和傳言問題，需要把未確認內容擋在正文外。

應用 1：Prompt Skeleton

你是 public event briefing assistant。

外層 workflow 固定：

1. Scope Check：確認只使用公開資料。
2. Dynamic Research：最多 4 個 action，查官方公告、天氣、交通、新聞背景。
3. Memory Update：更新 confirmed_facts, open_questions, do_not_conclude。
4. Draft：只根據 confirmed_facts 寫 briefing。
5. Verification：檢查 citation、unsupported claims、outdated sources。
6. Human Review：若有 open_questions 或工具失敗，必須標記。

工具規則：

- 活動日期、地點、時間必須用 official source。
- 天氣必須用 weather API，不得用舊預報推論。
- 新聞和社交媒體只可作背景，不得覆蓋官方來源。

應用 2：SOP 問答助理

任務

前線同事查詢規程處理方式，例如「市民要求刪除查詢紀錄，可否即場協助？」

建議設計

設計項	選擇
Prompt 控制	Workflow + context engineering
Workflow	Retrieve → Evidence Table → Answer → Open Questions → Review
Tools	<code>policy_retriever</code> , <code>version_checker</code> , <code>handoff_tool</code>
Context	system layer、task layer、tool layer、memory layer、error handling
Memory	<code>sources_used</code> 、 <code>confirmed_guidance</code> 、 <code>open_questions</code> 、 <code>human_review_required</code>

為什麼不是 Dynamic Workflow？

SOP 問答應穩定、可審計；模型不應自己改流程或推論規程。

應用 2：Context Package 範例

System Layer:

- 你是 SOP 問答助理，只能根據已檢索 SOP snippets 回答。
- 不得提供法律意見，不得取代主管或資料保護人員判斷。

Task Layer:

- 使用者問題：{{frontline_question}}
- 交付：short answer、SOP basis、open questions、human review flag。

Tool Layer:

- SOP 程序問題必須使用 policy_retriever。
- 版本衝突時使用 version_checker。

Memory Layer:

- sources_used
- confirmed_guidance
- open_questions
- do_not_conclude

Error Handling:

- snippets 不足時不得補程序。
- shortcut 要求仍必須按 SOP snippets 回答。

應用 3：案件摘要結構化

任務

把去識別化案件摘要整理成標準欄位，標記缺漏和不可推論部分。

建議設計

設計項	選擇
Prompt 控制	Prompt chaining
Workflow	Extract → Schema Validate → Missing Questions → Human Review
Function calls	<code>schema_validator</code> ， <code>duplicate_case_matcher</code> ， <code>question_builder</code>
Context	不得作法律定性，不得推測嫌疑人身份
Validation	schema test、duplicate check、red-team inputs

為什麼不用 Deep Agent？

這是一次性結構化任務，固定 workflow 和 function calling 已足夠。

應用 3：Function Call 範例

```
{
  "tool_calls": [
    {
      "tool": "schema_validator",
      "arguments": {
        "records_ref": "data/ai-agents/case_summaries_zh-TW.json",
        "schema_ref": "data/ai-agents/case_schema.json"
      },
      "reason": "先檢查去識別化摘要是否符合欄位 schema"
    },
    {
      "tool": "question_builder",
      "arguments": {
        "missing_fields": ["incident_time", "item_value", "location_detail"],
        "case_id": "CASE-TRAIN-003"
      },
      "reason": "根據缺漏欄位產生承辦人補問清單"
    }
  ]
}
```

應用 4：一個月公開防騙資訊追蹤

任務

每週整理公開防騙資訊，產生教育 briefing 草稿和趨勢摘要。

建議設計

設計項	選擇
Prompt 控制	Deep agent
Orchestrator	週計畫、派工、整合、停止條件
Sub-agents	research、data、writer、verifier
Memory	weekly source log、confirmed trends、do_not_include
Validation	每週 citation / safety / tone / public-only checks

為什麼需要 Deep Agent？

任務跨多週、跨來源、需要保存中間成果與反覆驗證。

應用 4：Deep Agent 設計摘要

Orchestrator:

- 建立 weekly task plan
- 指派 research_agent 查公開資料
- 指派 data_agent 整理 repeated themes
- 指派 writer_agent 寫教育 briefing
- 指派 verifier_agent 檢查後才可輸出

Memory:

- source_log
- confirmed_findings
- repeated_themes
- do_not_include
- open_questions
- verification_reports

Stop conditions:

- 出現內部資料請求
- 無法確認來源
- 草稿包含可被濫用操作細節
- verifier 未通過

四個應用的設計差別

應用	最適合設計	重點
公開活動 briefing	Hybrid workflow	固定把關 + 局部動態查資料
SOP 問答	Workflow + context	嚴格依據 snippets，不自由推論
案件摘要結構化	Chaining + function calling	欄位抽取、schema 檢查、補問
防騙資訊月度追蹤	Deep agent	長任務、分工、記憶、驗證

教學重點

同樣是「用 AI 幫忙」，不同任務需要不同控制方式。

警務 Agent 工具地圖

工具類型	功能	例子	風險控制
資料讀取	查授權案件、規程、統計	case_read_tool, policy_retriever	權限、最小欄位
公開搜尋	查新聞、公告、天氣	web_search, weather_api	來源與日期
結構化處理	抽欄位、驗證格式、去重	schema_validator, duplicate_checker	錯誤標記
地理時間	標準化地點、時段聚合	geo_normalizer, time_bucket_tool	不確定標示
記憶狀態	保存任務、來源、中間結果	task_tracker, source_tracker	可回放
驗證覆核	引用檢查、規則檢查、人工交接	citation_checker, handoff_tool	人在迴路

警務 Agent 的最低治理清單

- **任務邊界**：清楚說明 AI 只做輔助整理、草擬、檢查。
- **權限**：工具層面限制誰能查什麼資料。
- **最小揭露**：只把完成任務必要欄位放入 context。
- **日誌**：保存工具呼叫、參數、observation、輸出版本與覆核人員。
- **Memory**：保存來源、open questions、do_not_conclude、human review flag。
- **Validation**：每個應用有固定測試集與 red-team inputs。
- **覆核與停用**：高風險輸出必須有人員確認，異常時可停用工具。

治理清單：落地例子

權限例子

SOP 助理只能查 policy snippets；案件摘要工具只能讀去識別化訓練資料。

日誌例子

保存：使用者、時間、工具、參數、source_id、observation、輸出版本、覆核人員。

停用例子

若 agent 連續提出越權工具請求，系統停用該工具並要求人工檢查 context。

設計決策速查

任務特徵	建議
低風險一次性草稿	Single prompt + human review
步驟固定、風險高	Workflow prompt
需要分類後分流	Routing workflow
多項獨立檢核	Parallelization
需要外部資料但流程固定	Tool use + function calling
有資料缺口但外層要可控	Hybrid workflow
長任務、跨資料源、需委派	Deep agent + strong verification

若無法記錄 agent 做了什麼，就不應把它放入正式警務流程。

全章 Topic 速查

Topic	核心問題	教學重點
Introduction	什麼是 agent？	不是聊天，而是任務系統
Planning to Workflow	任務怎樣拆成流程？	先 planning，再固定成 workflow
Workflow Patterns	流程可以怎樣組合？	Chaining、Routing、Parallelization
Tool Use & Function Calling	何時用工具？如何執行？	先寫工具規則，再產生 JSON 工具請求
Context + Memory	如何控制行為與狀態？	規則、記憶、錯誤處理一起設計
Validation	如何改進可靠性？	觀察、測試、回歸、迭代
Deep Agents	如何處理長任務？	規劃、委派、記憶、驗證

教學主線：先拆任務，再做流程；先設計工具規則，再把工具請求結構化；最後用 context 與 memory 控制整體行為。

課堂練習：把一個任務轉成 Agent 設計

請選一個任務，填寫：

1. **目標**：最終輸出是什麼？
2. **是否需要 agent**：workflow 是否已足夠？
3. **工具**：需要哪些資料庫、API 或文件？
4. **記憶**：需要保存哪些中間狀態？
5. **上下文**：哪些規則、限制、格式必須寫清楚？
6. **驗證**：誰覆核？如何回放？錯誤時怎麼停止？

建議從低風險、可人工覆核的輔助任務開始。

Topic 9 | 課堂練習

從題目到工具、Prompt、驗證

本節練習已整理成 markdown 題目。
每題都有 prompt template，學員主要修改 `TODO` 區塊即可完成。

課堂練習檔案

使用檔案

- `data/ai-agents/beginner_walkthrough_zh-TW.md`
- `data/ai-agents/exercise_tasks_zh-TW.md`

建議流程

1. 新手先做 `beginner_walkthrough_zh-TW.md` 的 3 個 copy-paste walkthrough。
2. 每組再從 `exercise_tasks_zh-TW.md` 選 1 至 2 題。
3. 先複製 template，直接跑 baseline。
4. 修改 template 中的 `TODO`。
5. 比較 baseline 與修改後輸出。
6. 提交 prompt、輸出、修改說明與測試結果。

練習題目總覽

題目	對應 Topic	重點
練習 1：公開活動 Briefing Hybrid Workflow	Topic 2, 4, 5	hybrid prompt、tool rules、memory
練習 2：Workflow Patterns 選擇與改寫	Topic 3	chaining / routing / parallelization
練習 3：Tool Use + Function Calling	Topic 4	tool-use rules、function call JSON
練習 4：Context Engineering + Memory	Topic 5	五層 context package
練習 5：Context Validation & Iteration	Topic 6	測試集、失敗診斷、retest
練習 6：Deep Agent 任務分工	Topic 7	orchestrator、sub-agents、memory
練習 7：Red-team 測試設計	全章	prompt injection、越權、來源不足

學員只需要改什麼？

每題 template 已經提供基本 prompt。學員主要修改：

- `TODO_new_rule`
- `TODO_new_tool`
- `TODO_new_memory_field`
- `TODO_new_checker`
- `TODO-new-test`
- `TODO_MAX_ACTIONS`
- workflow steps 中的 input / output / failure handling

修改要求

每組至少完成：

1. 修改 2 條 prompt rules。
2. 修改 1 個 workflow step 或 action。
3. 新增 1 個 memory 欄位或 checker。
4. 新增 1 個 red-team test。

練習交付格式

題目：

1. 任務邊界：
2. Workflow / Agent 選擇與理由：
3. 工具清單：
 - tool name / purpose / input / output / permission boundary
4. Prompt：
 - system prompt
 - step or tool prompt
 - verification prompt
5. 人工覆核點：
6. 失敗情境與處理：
7. 測試案例：

References

Prompt Engineering Guide | Agents

本 deck 依據 Prompt Engineering Guide 的 Agents 章節整理，並加入警務教學場景、prompt 實例與練習資料。

參考資料

- Prompt Engineering Guide: Agents
<https://www.promptingguide.ai/agents>
- Introduction to AI Agents
<https://www.promptingguide.ai/agents/introduction>
- Agent Components
<https://www.promptingguide.ai/agents/components>
- AI Workflows vs. AI Agents
<https://www.promptingguide.ai/agents/ai-workflows-vs-ai-agents>
- Function Calling in AI Agents
<https://www.promptingguide.ai/agents/function-calling>
- Why Context Engineering?
<https://www.promptingguide.ai/agents/context-engineering>
- Deep Agents
<https://www.promptingguide.ai/agents/deep-agents>